

Resumo do projeto mult-agent

1. O que é o projeto

mult-agent é uma aplicação de linha de comando (CLI) de pesquisa assistida por um modelo de linguagem (LLM). A partir de um tema digitado pela pessoa usuária, a ferramenta:

1. busca páginas na web (mecanismo DuckDuckGo);
2. baixa cada página e extrai o texto principal;
3. envia esse texto para um LLM executado localmente na máquina (via Ollama, modelo qwen2.5:7b) para resumo;
4. combina os resumos numa síntese única com citações numeradas;
5. gera as referências bibliográficas no padrão IEEE;
6. salva um relatório em .txt com cabeçalho de auditoria e registra a consulta num log de histórico.

2. Objetivo de estudo

O projeto foi construído para investigar a fronteira entre dois paradigmas de construção de sistemas com LLM:

Paradigma	Quem decide a sequência de passos	Como é chamado na literatura
Workflow / pipeline	o código (for/if fixos escritos pela pessoa desenvolvedora)	<i>workflow</i> , pipeline determinístico
Agente autônomo	o próprio modelo, num laço de <i>tool-calling</i>	<i>agent</i>

Por isso a aplicação oferece dois modos de execução, selecionados num menu inicial, que produzem o mesmo tipo de saída mas por caminhos conceitualmente diferentes.

Modo 1 — Pipeline determinístico

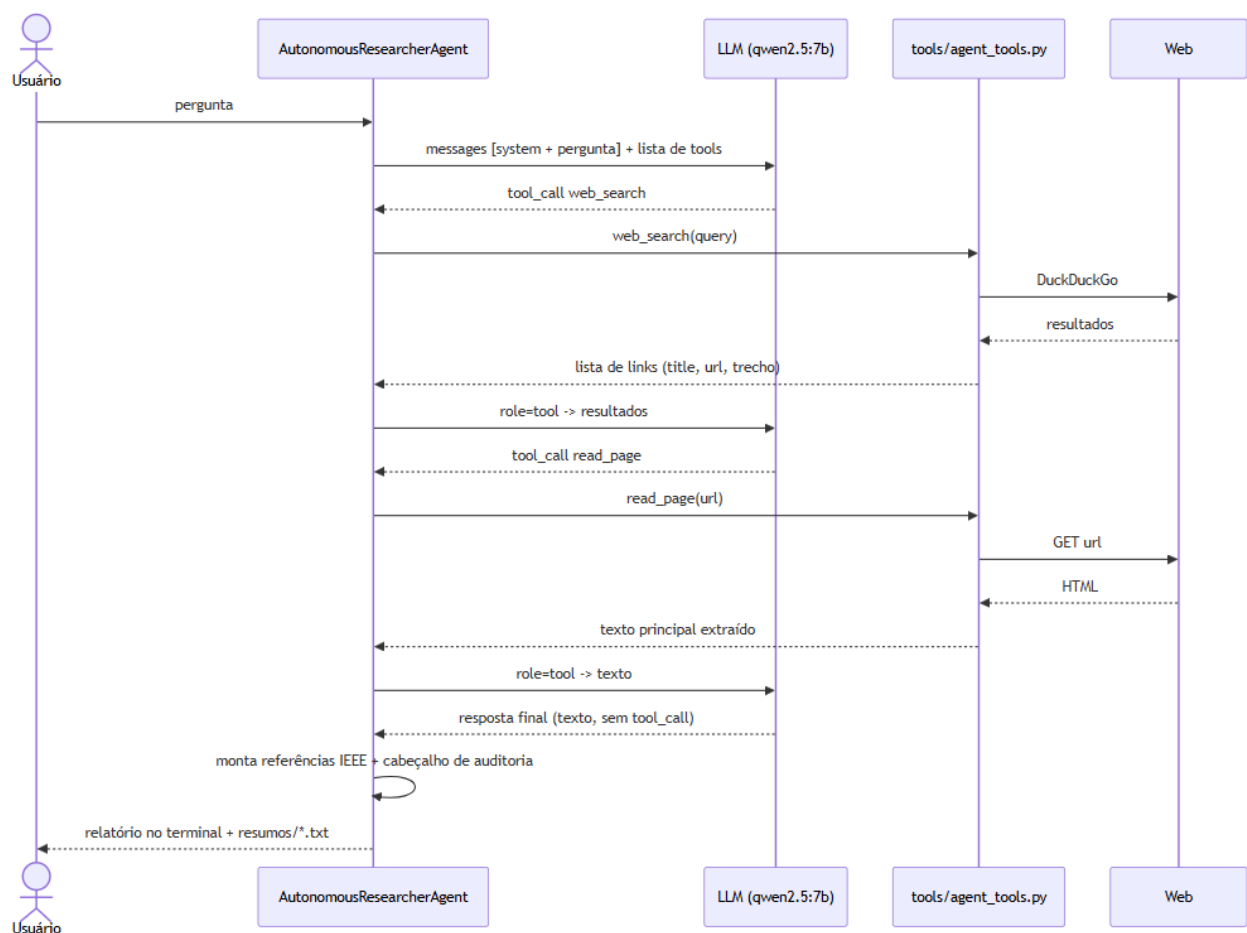
O fluxo é fixo e escrito em código. O LLM é chamado apenas para tarefas pontuais (moderar um tema, resumir uma página, sintetizar os resumos) e nunca decide qual é o próximo passo. A sequência é sempre: buscar → ler N páginas → resumir cada uma → sintetizar → montar referências → salvar.

Modo 2 — Agente autônomo

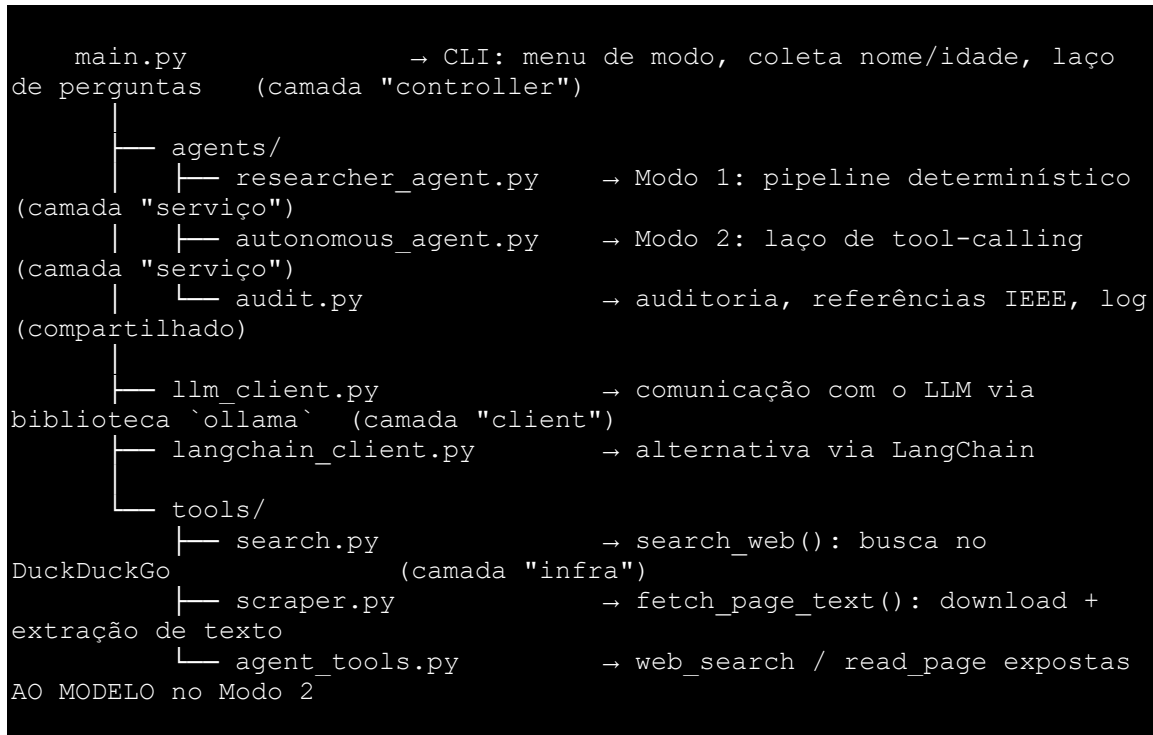
Existe um laço de decisão: a cada rodada, o modelo recebe a lista de ferramentas disponíveis (`web_search`, `read_page`) e responde uma de duas coisas:

- *"chame a ferramenta X com estes argumentos"* → o código executa a função Python correspondente, devolve o resultado ao modelo e o laço continua;
- um texto normal, sem pedir ferramenta → é a resposta final e o laço termina.

Quem decide se busca, o que busca, quantas vezes, quais páginas ler e quando já tem informação suficiente é o modelo. O laço de *tool-calling* foi escrito à mão (cerca de 50 linhas) em vez de usar um framework, para fins de estudo — é o mesmo mecanismo que `AgentExecutor` / `create_react_agent` do LangChain implementam internamente.



O código é organizado em camadas, com uma regra de dependência unidirecional (análoga a uma arquitetura em camadas de aplicação Java):



Regra de dependência: `tools/` não conhece `agents/`; `agents/` conhece `tools/` e os clients; `main.py` só conhece `agents/`.

A saída é gravada na pasta `resumos/` (criada em tempo de execução):

- `resumos/<timestamp>_<tema>.txt` — um arquivo por pesquisa concluída, contendo cabeçalho de auditoria + síntese + referências IEEE;
- `resumos/historico_auditoria.log` — uma linha para toda tentativa de pesquisa, inclusive as bloqueadas.

4. Decisões de projeto relevantes

- **Segurança não é delegada ao modelo.** Há um filtro determinístico para pessoas usuárias menores de 18 anos: antes de qualquer busca, nos dois modos, o tema é submetido a uma verificação de adequação. Se reprovado, a pesquisa é bloqueada e registrada no log.

- **As referências IEEE são montadas em Python puro**, a partir das páginas que realmente passaram pela função de leitura — não pelo LLM. Modelos de linguagem costumam "citar" URLs inexistentes; essas são descartadas.
- **Rastreabilidade / auditoria.** Todo relatório traz um cabeçalho com pessoa usuária, idade, pergunta feita, data/hora, navegador padrão do sistema, mecanismo de busca, modelo LLM e versão do agente. No Modo 2 há linhas extras (número de rodadas de decisão, de buscas e de páginas lidas).
- **Freios determinísticos no Modo 2.** Como o qwen2.5:7b é um modelo pequeno, às vezes ele ignora as ferramentas e tenta responder "de memória". Há dois mecanismos de contenção (um lembrete injetado e uma leitura automática das primeiras páginas) para garantir que a resposta final seja fundamentada em fontes reais. Com um modelo maior, ou via API, esses freios quase não disparam.

5. Tecnologias utilizadas

Componente	Ferramenta
Linguagem	Python 3.12
LLM local	Ollama + modelo qwen2.5:7b
Busca web	DuckDuckGo (biblioteca ddgs)
Extração de texto	trafilatura, com <i>fallback</i> puro-Python (html.parser + httpx)
Interface de terminal	rich
Framework de agente (alternativo)	langchain-core / langchain-ollama

6. Como executar

Pré-requisito: o aplicativo Ollama em execução (servidor local em localhost:11434) com o modelo baixado (ollama pull qwen2.5:7b).

```
# instalar as dependências (uma vez)
.venv\Scripts\python.exe -m pip install -r requirements.txt

# executar
.venv\Scripts\python.exe main.py
```

No menu inicial, escolha o modo **1** (pipeline determinístico) ou **2** (agente autônomo). No modo 2, cada ferramenta que o modelo decide chamar aparece em destaque no terminal, em tempo real.

7. Limitações conhecidas do ambiente

O computador de desenvolvimento tem o Smart App Control do Windows ativo, que bloqueia DLLs nativas não assinadas instaladas via pip. Isso afeta:

- `uuid_utils` (dependência do `langchain-core`) → o caminho `LangChain` do Modo 1 falha no `import`; o `main.py` detecta e volta automaticamente para o Ollama direto;
- `lxml` (dependência do `trafilatura`) → por isso o extrator de texto tem um *fallback* puro-Python, de qualidade um pouco menor mas funcional.

O modelo `qwen2.5:7b` é pequeno; um modelo maior (local ou via API) melhora a qualidade das sínteses e a adesão ao uso de ferramentas no Modo 2.